

Menu et arbres

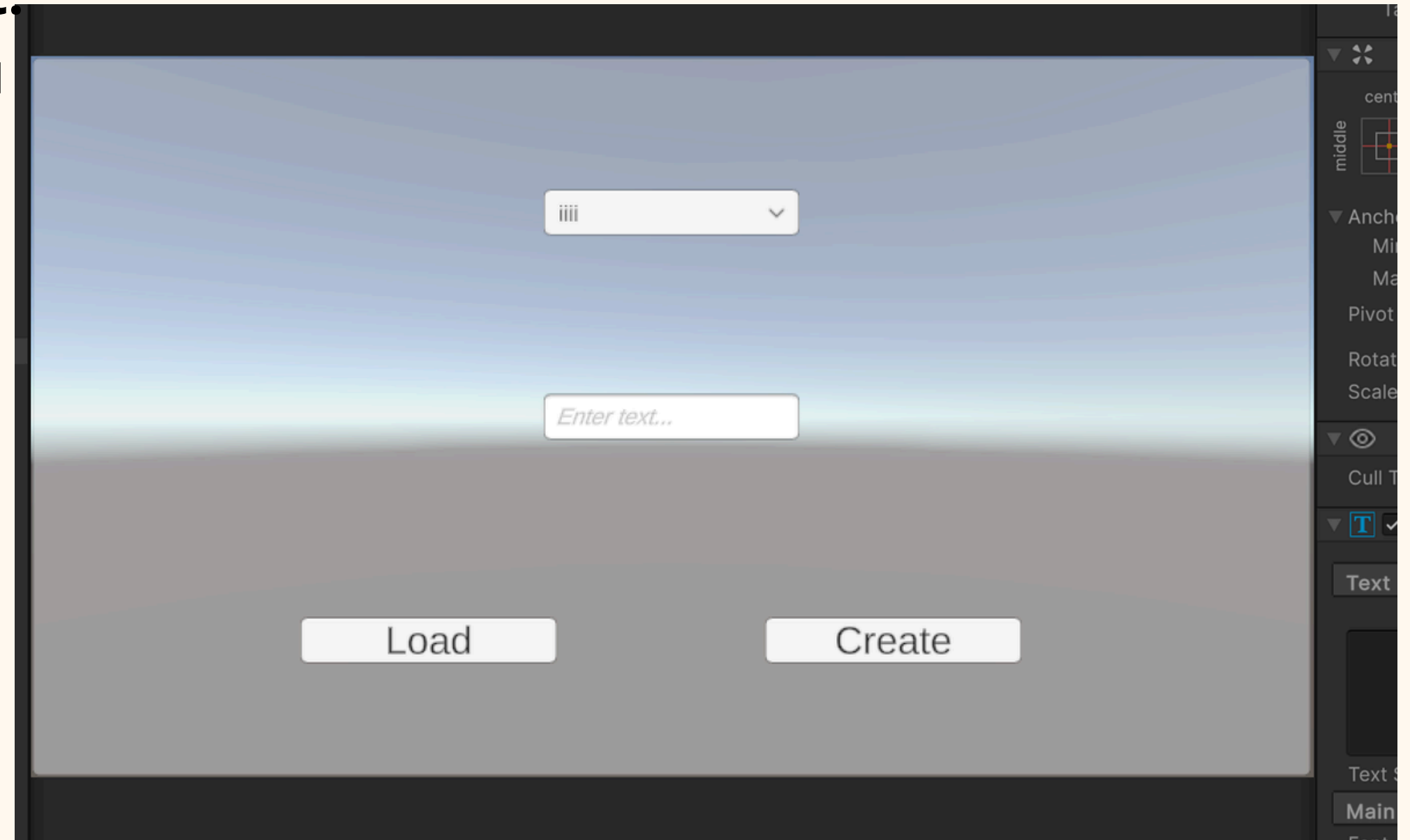


Objectif

- **Aujourd'hui tu ajoutes un menu de mondes dans ton jeu.**
- **Le joueur peut choisir un monde existant ou en créer un nouveau.**
- **Le nom saisi devient l'ID du monde et crée un fichier JSON séparé.**

Résultat Final Attendu

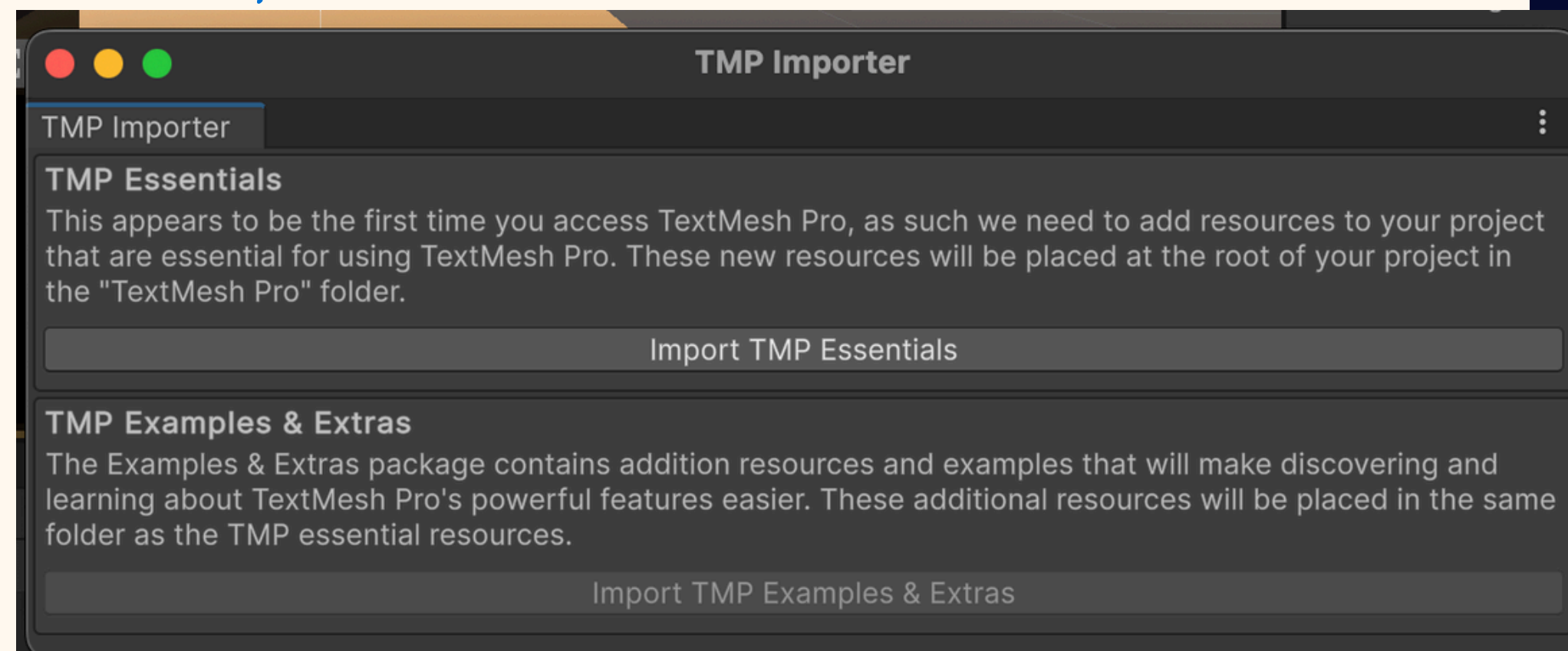
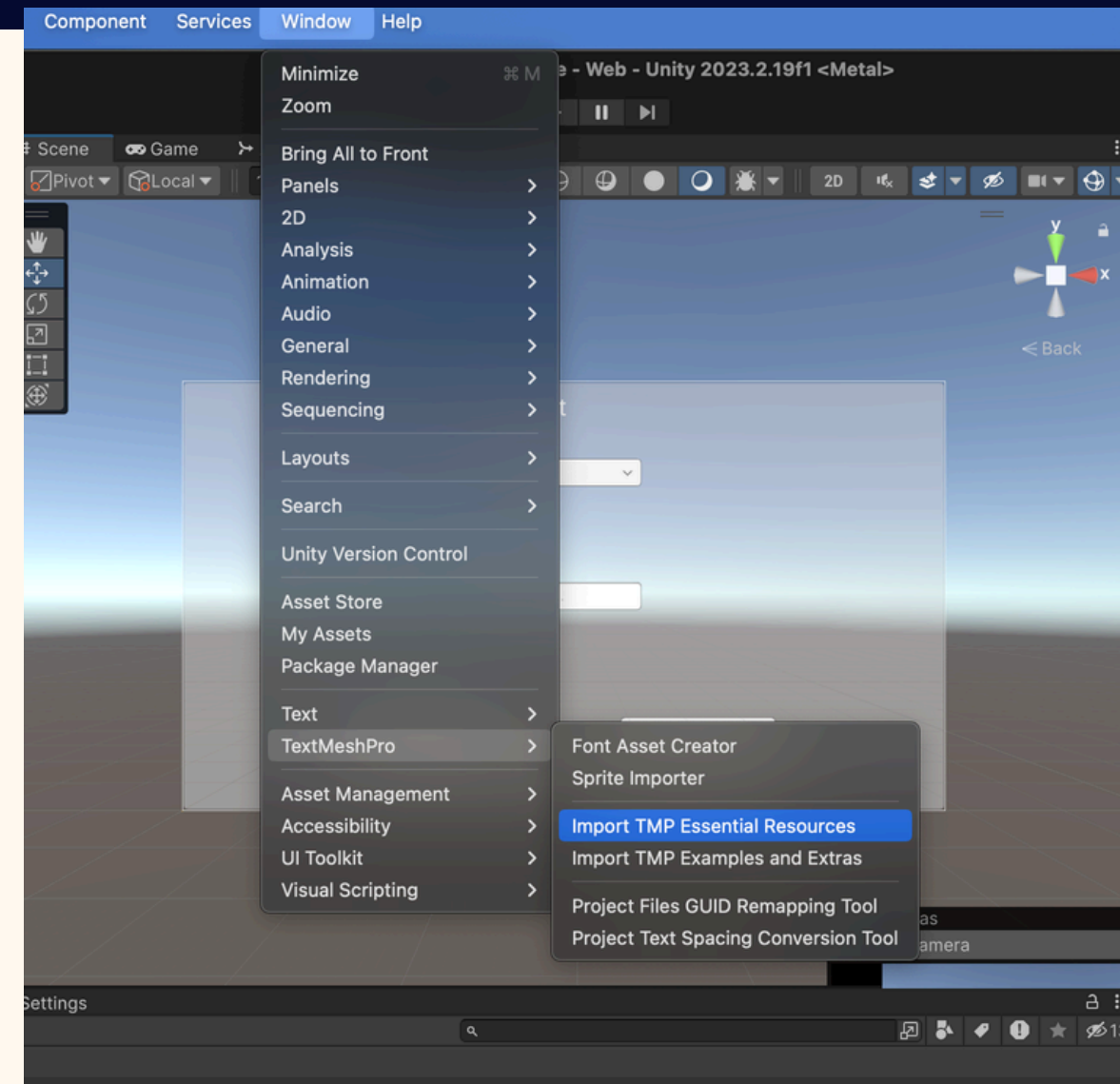
- Au lancement, le jeu montre un menu.
- Bouton 1: Charger un monde existant.
- Bouton 2: Créer et lancer un nouveau monde.
- Chaque monde est un fichier `worlds/<id>.json`.



PS : N'hésite pas à faire un rendu plus joli que celui-ci. Tu peux ajouter des couleurs, un arrière-plan, modifier la police, etc.

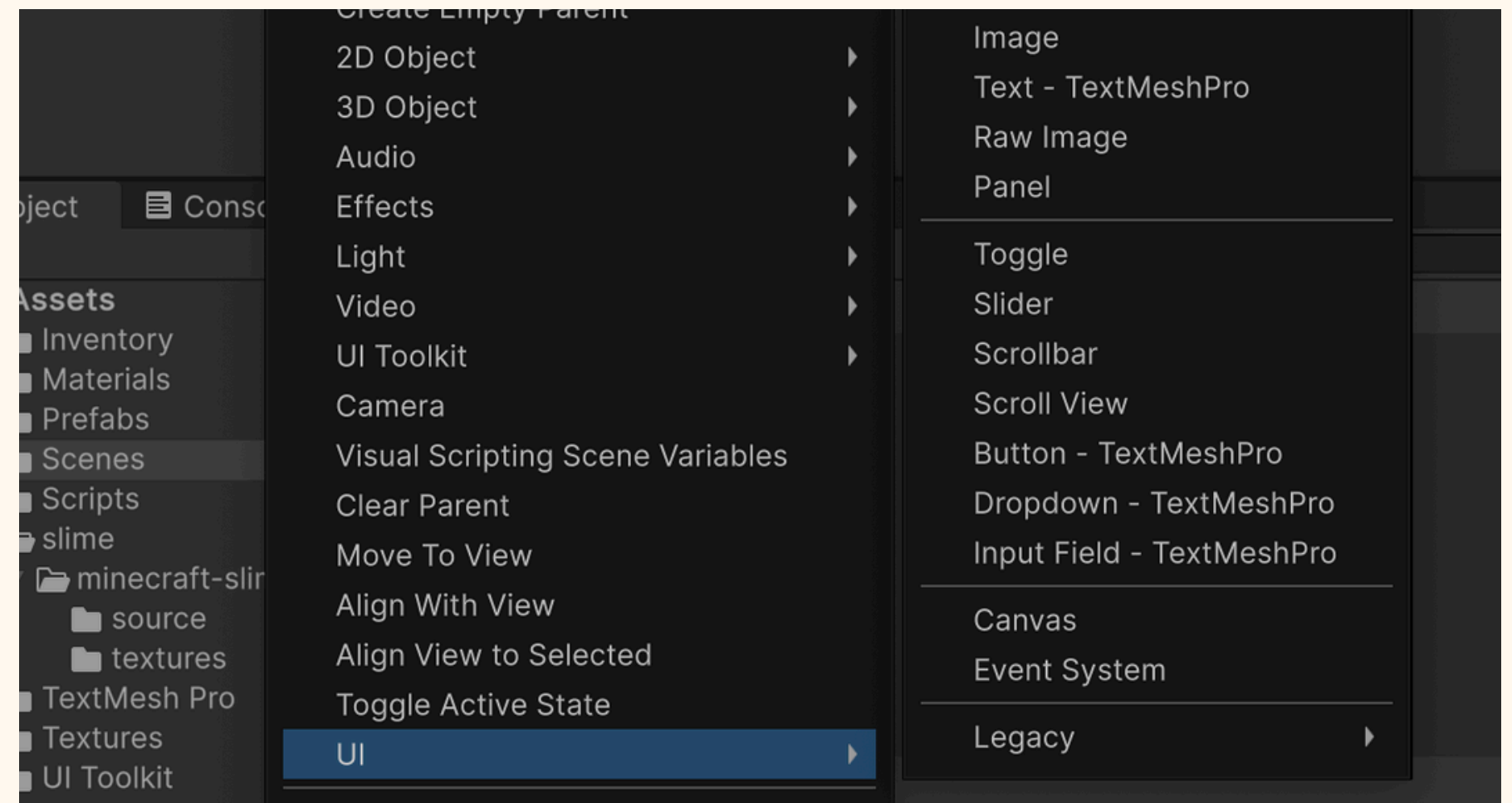
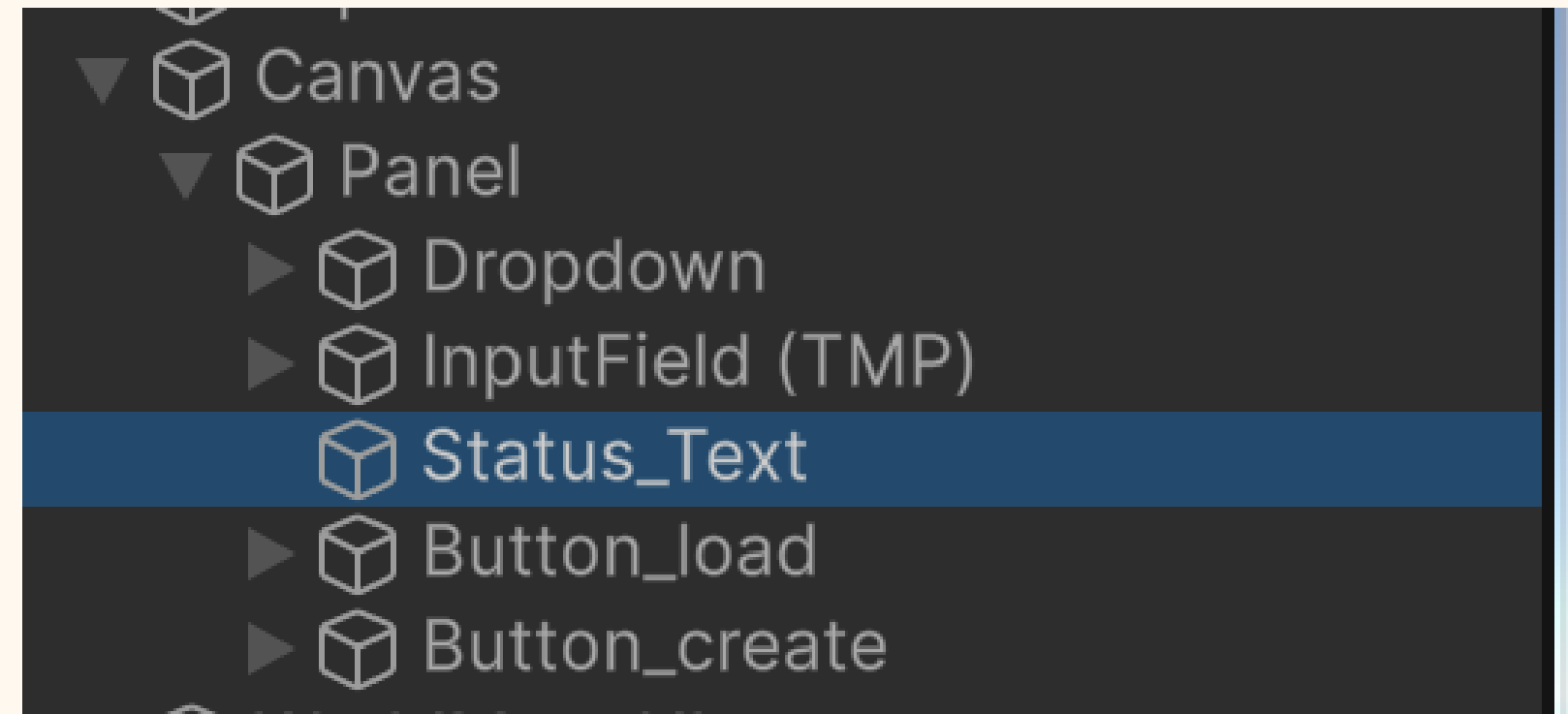
Pré-requis:

- Importer TMP si ce n'est pas déjà fait.
- Window → TextMeshPro → Import TMP Essentials



Créer L'UI Dans La Scène

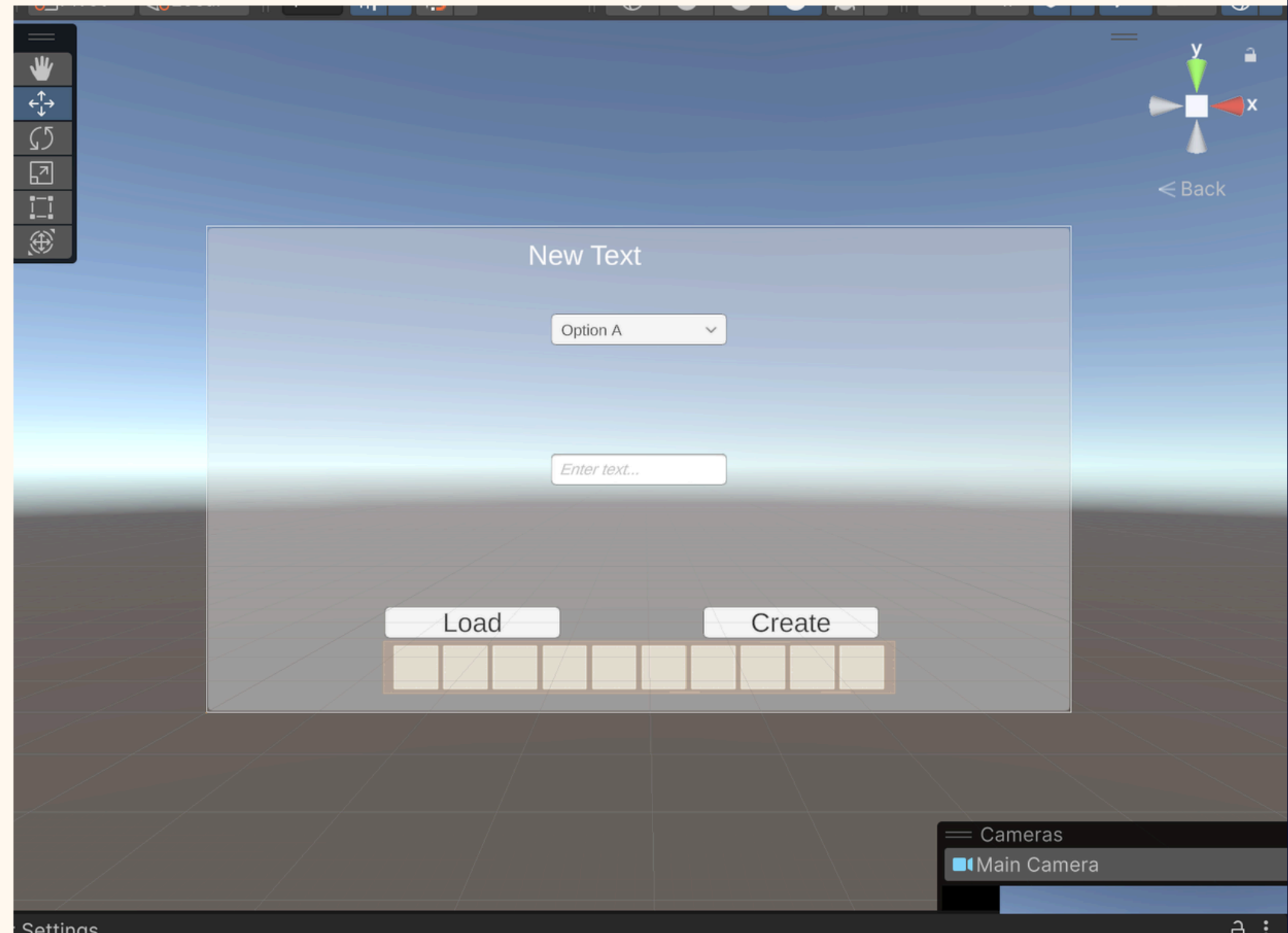
- Dans la hiérarchie, créer:
 - Canvas
 - Panel
 - Dropdown (TMP_Dropdown)
 - InputField (TMP)
 - Status_Text (TextMeshPro - Text)
 - Button_load
 - Button_create
- Renommer exactement comme ci-dessus.
- tous ces éléments se trouve dans l'onglet UI



Faire Un Menu Lisible

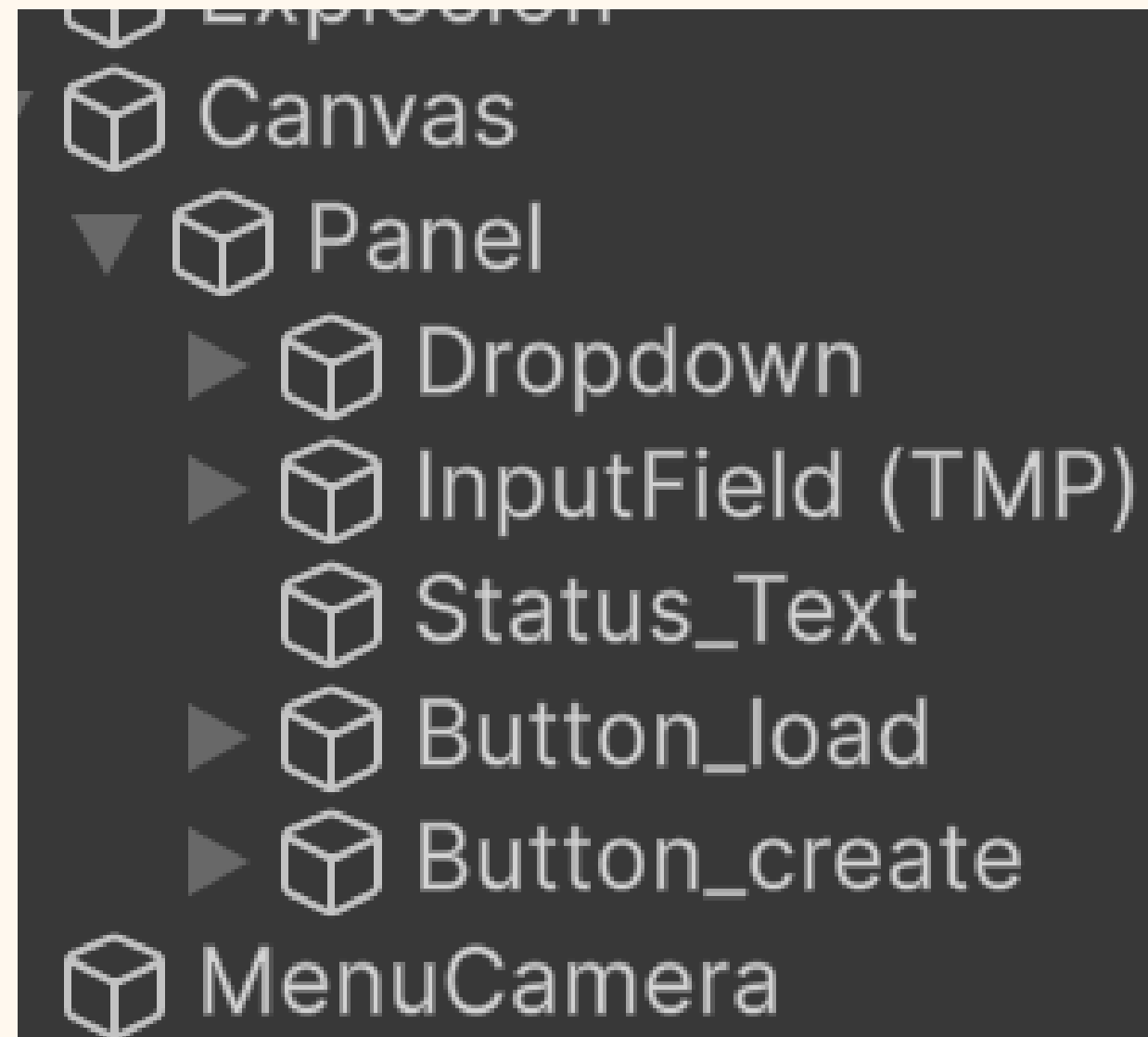
- Dans le Panel:
 - Place le Dropdown en haut.
 - Place InputField au milieu.
 - Place les 2 boutons en bas.
 - Change le texte des boutons:
 - Charger
 - Créer

PS : N'hésite pas à faire un rendu plus joli que celui-ci. Tu peux ajouter des couleurs, un arrière-plan, modifier la police, etc.



Ajouter Une Caméra De Menu

- Créer un nouvel objet Camera nommé MenuCamera.
- Laisser la caméra du joueur dans le Player.



Créer Le Script Du Menu

- Créer un script: WorldSelectionMenuUI
- Ce script contrôle tout le menu: liste des mondes, création, chargement, messages.
- Dans un premier temps, on prépare les références Unity et TMP.
- ajoute ces variables d'instances

```
[Header("References")]  
10 references | Unity Serialized Field  
public WorldManagerScript worldManager;  
8 references | Unity Serialized Field  
public Camera menuCamera;  
9 references | Unity Serialized Field  
public GameObject panel;  
7 references | Unity Serialized Field  
public TMP_Dropdown worldsDropdown;  
2 references | Unity Serialized Field  
public TMP_InputField newWorldInput;  
2 references | Unity Serialized Field  
public TMP_Text textStatus;  
  
2 references  
private const string NoWorldLabel = "(aucun monde)";
```


SetStatus()

Affiche un message s'il y a un problème, par exemple si le nom du monde est invalide.

6 references

```
private void SetStatus(string message)
{
    if (statusText != null)
        statusText.text = message;

    if (!string.IsNullOrEmpty(message))
        Debug.Log(message);
}
```

Start()

- Start() prépare le menu au lancement du jeu.
- Si un monde est déjà lancé, on cache le menu. Sinon on affiche le menu et on libère la souris.

```
0 references | Unity Message
void Start()
{
    if (worldManager == null)
        worldManager = FindFirstObjectByType<WorldManagerScript>();

    if (worldManager != null && worldManager.IsWorldInitialized)
    {
        if (menuCamera != null)
            menuCamera.enabled = false;

        if (panel != null)
            panel.SetActive(false);
        return;
    }

    if (menuCamera != null)
        menuCamera.enabled = true;

    if (panel != null)
        panel.SetActive(true);

    RefreshWorldList();
    SetStatus("");
    ForceMenuCursor();
}
```

Update() et ForceMenuCursor()

- On force le mode menu tant que le monde n'est pas lancé.
- `panel.activeSelf` dit si le panel est visible.
- Cette boucle empêche le script FPS de reprendre la souris pendant le menu.

0 references | Unity Message

```
void Update()
{
    if (!worldManager.IsWorldInitialized)
    {
        if (menuCamera != null)
            menuCamera.enabled = true;

        if (panel != null && !panel.activeSelf)
            panel.SetActive(true);
    }

    ForceMenuCursor();
}
```

2 references

```
private void ForceMenuCursor()
{
    Cursor.lockState = CursorLockMode.None;
    Cursor.visible = true;
}
```

RefreshWorldList()

- Cette fonction remplit la liste déroulante avec les mondes trouvés. (les noms des fichiers json)
- List<string> = liste de textes.
- ids.Count = nombre d'éléments.
- ClearOptions() nettoie l'ancienne liste.
- AddOptions(ids) ajoute les nouveaux noms.
- on implémentera la fonction GetAvailableWorldIds() plus tard dans de WorldManager.

```
1 reference
public void RefreshWorldList()
{
    List<string> ids = WorldManagerScript.GetAvailableWorldIds();
    worldsDropdown.ClearOptions();

    if (ids.Count == 0)
        ids.Add(NoWorldLabel);

    worldsDropdown.AddOptions(ids);
}
```

LoadSelectedWorld()

Code qui s'exécutera quand on cliquera sur le bouton « Charger ».

Récupère l'élément sélectionné dans le dropdown.

`worldsDropdown.value = index choisi`
`options[index].text = texte affiché`

```
0 references
public void LoadSelectedWorld()
{
    //aucun monde crée donc 0 élément dans le dropdown
    if (worldsDropdown == null || worldsDropdown.options.Count == 0)
    {
        SetStatus("Pas de monde a charger.");
        return;
    }

    string id = worldsDropdown.options[worldsDropdown.value].text;
    if (id == NoWorldLabel)
    {
        SetStatus("Pas de monde a charger.");
        return;
    }

    StartWorld(id);
}
```


CreateAndStartWorld()

Code qui s'exécutera quand on cliquera sur le bouton « Créer »

- `?:` = opérateur ternaire (version courte d'un if).
- `NormalizeWorldId` supprime les espaces inutiles.
- `IsValidWorldId` protège les noms de fichiers.
- `Contains` vérifie si le monde existe déjà.

```
public void CreateAndStartWorld()
{
    string id = newWorldInput != null ? newWorldInput.text : "";
    id = WorldManagerScript.NormalizeWorldId(id);

    if (!WorldManagerScript.IsValidWorldId(id))
    {
        SetStatus("Nom invalide. Utilise lettres/chiffres/_/-");
        return;
    }

    List<string> worlds = WorldManagerScript.GetAvailableWorldIds();
    if (worlds.Contains(id))
    {
        SetStatus("Ce monde existe deja.");
        return;
    }

    StartWorld(id);
}
```

StartWorld()

Lance vraiment la partie.

- SelectWorld enregistre l'ID choisi.
- StartSelectedWorld initialise le monde côté gameplay.
- Puis on ferme le menu et on rend le contrôle souris en mode FPS.

```
private void StartWorld(string worldId)
{
    WorldManagerScript.SelectWorld(worldId);
    worldManager.StartSelectedWorld();

    if (panel != null)
        panel.SetActive(false);

    if (menuCamera != null)
        menuCamera.enabled = false;

    Cursor.lockState = CursorLockMode.Locked;
    Cursor.visible = false;
    SetStatus("");
}
```


World script manager

Dans WorldManager, on va faire 4 changements pour passer à un système “plusieurs mondes” :

- 1. Stocker le monde actuellement choisi (CurrentWorldId).**
- 2. Construire un chemin de sauvegarde différent par monde (worlds/<id>.json).**
- 3. Empêcher le jeu de démarrer tant qu’aucun monde n’est choisi.**
- 4. Charger/créer le monde sélectionné puis sauvegarder proprement.**

Nouvelles variables (dans WorldManagerScript.cs)

- On passe d'un seul save.json à un fichier JSON par monde.
- On ajoute des variables pour savoir:
- quel monde est choisi, si le monde est lancé, où spawn le joueur au premier lancement.

```
4 references
public static string CurrentWorldId { get; private set; }
4 references
public static string WorldsDirectory => Path.Combine(Application.persistentDataPath, "worlds");
4 references
private string SavePath => GetWorldFilePath(CurrentWorldId);
2 references
public bool IsWorldInitialized => worldInitialized;

[Header("Références")]
11 references | ☑ Unity Serialized Field
public WorldGenerator generator;
13 references | ☑ Unity Serialized Field
public GameObject player;
7 references | ☑ Unity Serialized Field
public ChunkSpawner chunkSpawner;
```

Fonctions d'ID de monde

- Ces fonctions gèrent le nom du monde
- Trim() retire les espaces au début et à la fin.
- IsNullOrWhiteSpace refuse vide ou "que des espaces".
- foreach (char c in worldId) regarde caractère par caractère.
- On autorise seulement lettres/chiffres/_/- pour éviter des noms de fichiers problématiques.
- SelectWorld est la "mémoire" du monde actuellement choisi.

```
1 reference
public static void SelectWorld(string worldId)
{
    CurrentWorldId = NormalizeWorldId(worldId);
}

3 references
public static bool IsValidWorldId(string worldId)
{
    if (string.IsNullOrEmpty(worldId))
        return false;

    foreach (char c in worldId)
    {
        if (char.IsLetterOrDigit(c))
            continue;

        if (c == '_' || c == '-')
            continue;

        return false;
    }

    return true;
}

2 references
public static string NormalizeWorldId(string worldId)
{
    return worldId == null ? string.Empty : worldId.Trim();
}
```

Lire les mondes existants + construire le chemin du fichier

- Ces fonctions lisent les .json déjà présents dans le dossier worlds.

Directory.Exists vérifie si le dossier existe.

Directory.GetFiles(..., "*.json") lit tous les fichiers json.

GetFileNameWithoutExtension transforme monde1.json en monde1.

Sort(...) trie la liste pour un menu plus propre.

safeId protège le code: si ID invalide, on utilise default.

2 references

```
public static List<string> GetAvailableWorldIds()
{
    List<string> ids = new List<string>();
    if (!Directory.Exists(WorldsDirectory))
        return ids;

    foreach (string path in Directory.GetFiles(WorldsDirectory, "*.json"))
        ids.Add(Path.GetFileNameWithoutExtension(path));

    ids.Sort(System.StringComparer.OrdinalIgnoreCase);
    return ids;
}
```

1 reference

```
public static string GetWorldFilePath(string worldId)
{
    string safeId = IsValidWorldId(worldId) ? worldId : "default";
    return Path.Combine(WorldsDirectory, safeId + ".json");
}
```

Nouveau Start()

Avant: le monde se lançait directement.

Maintenant: si aucun monde n'est sélectionné, on laisse le joueur désactivé (menu d'abord).

0 references | Unity Message

```
void Start()
{
    materialLookup.Clear();
    foreach (var entry in blockMaterials)
        materialLookup[entry.type] = entry.material;

    if (player != null)
    {
        startPlayerPosition = player.transform.position;
        startPlayerRotation = player.transform.rotation;
    }

    if (string.IsNullOrEmpty(CurrentWorldId))
    {
        SetGameplayEnabled(false);
        return;
    }

    StartSelectedWorld();
}
```

2 references

```
private void SetGameplayEnabled(bool enabled)
{
    if (player != null)
        player.SetActive(enabled);
}
```


Nouveau Start()

Avant: le monde se lançait directement.

Maintenant: si aucun monde n'est sélectionné, on laisse le joueur désactivé (menu d'abord).

0 references | Unity Message

```
void Start()
{
    materialLookup.Clear();
    foreach (var entry in blockMaterials)
        materialLookup[entry.type] = entry.material;

    if (player != null)
    {
        startPlayerPosition = player.transform.position;
        startPlayerRotation = player.transform.rotation;
    }

    if (string.IsNullOrEmpty(CurrentWorldId))
    {
        SetGameplayEnabled(false);
        return;
    }

    StartSelectedWorld();
}
```

2 references

```
private void SetGameplayEnabled(bool enabled)
{
    if (player != null)
        player.SetActive(enabled);
}
```


Nouvelle fonction StartSelectedWorld()

Cette fonction charge le monde sélectionné ou en crée un nouveau.

- worldInitialized empêche de relancer 2 fois.
- Directory.CreateDirectory est safe: crée si absent, ne casse rien si présent.
- File.Exists(SavePath) dit si le monde existe déjà.
- chunkSpawner.Initialize(...) charge le JSON ou crée des données neuves.
- save.player != null:
- vrai = ancien monde, on restaure la position sauvegardée.
- faux = nouveau monde, on utilise position de départ de la scène.
- if (!worldFileExists || save.player == null) SaveWorld();
- force la création immédiate du JSON pour un monde neuf.

```
2 references
public void StartSelectedWorld()
{
    if (worldInitialized)
        return;

    if (!IsValidWorldId(CurrentWorldId))
        return;

    Directory.CreateDirectory(WorldsDirectory);
    bool worldFileExists = File.Exists(SavePath);

    var save = chunkSpawner.Initialize(this, SavePath);
    Debug.Log("World file: " + SavePath);

    if (player != null)
    {
        if (save.player != null)
        {
            player.transform.position = save.player.position;
            player.transform.rotation = save.player.rotation;
        }
        else
        {
            player.transform.position = startPlayerPosition;
            player.transform.rotation = startPlayerRotation;
        }
    }

    worldInitialized = true;
    SetGameplayEnabled(true);

    if (!worldFileExists || save.player == null)
        SaveWorld();
}
```


Nouvelle fonction StartSelectedWorld()

Cette fonction charge le monde sélectionné ou en crée un nouveau.

- worldInitialized empêche de relancer 2 fois.
- Directory.CreateDirectory est safe: crée si absent, ne casse rien si présent.
- File.Exists(SavePath) dit si le monde existe déjà.
- chunkSpawner.Initialize(...) charge le JSON ou crée des données neuves.
- save.player != null:
- vrai = ancien monde, on restaure la position sauvegardée.
- faux = nouveau monde, on utilise position de départ de la scène.
- if (!worldFileExists || save.player == null) SaveWorld();
- force la création immédiate du JSON pour un monde neuf.

```
2 references
public void StartSelectedWorld()
{
    if (worldInitialized)
        return;

    if (!IsValidWorldId(CurrentWorldId))
        return;

    Directory.CreateDirectory(WorldsDirectory);
    bool worldFileExists = File.Exists(SavePath);

    var save = chunkSpawner.Initialize(this, SavePath);
    Debug.Log("World file: " + SavePath);

    if (player != null)
    {
        if (save.player != null)
        {
            player.transform.position = save.player.position;
            player.transform.rotation = save.player.rotation;
        }
        else
        {
            player.transform.position = startPlayerPosition;
            player.transform.rotation = startPlayerRotation;
        }
    }

    worldInitialized = true;
    SetGameplayEnabled(true);

    if (!worldFileExists || save.player == null)
        SaveWorld();
}
```

Sauvegarde sécurisée + état du monde

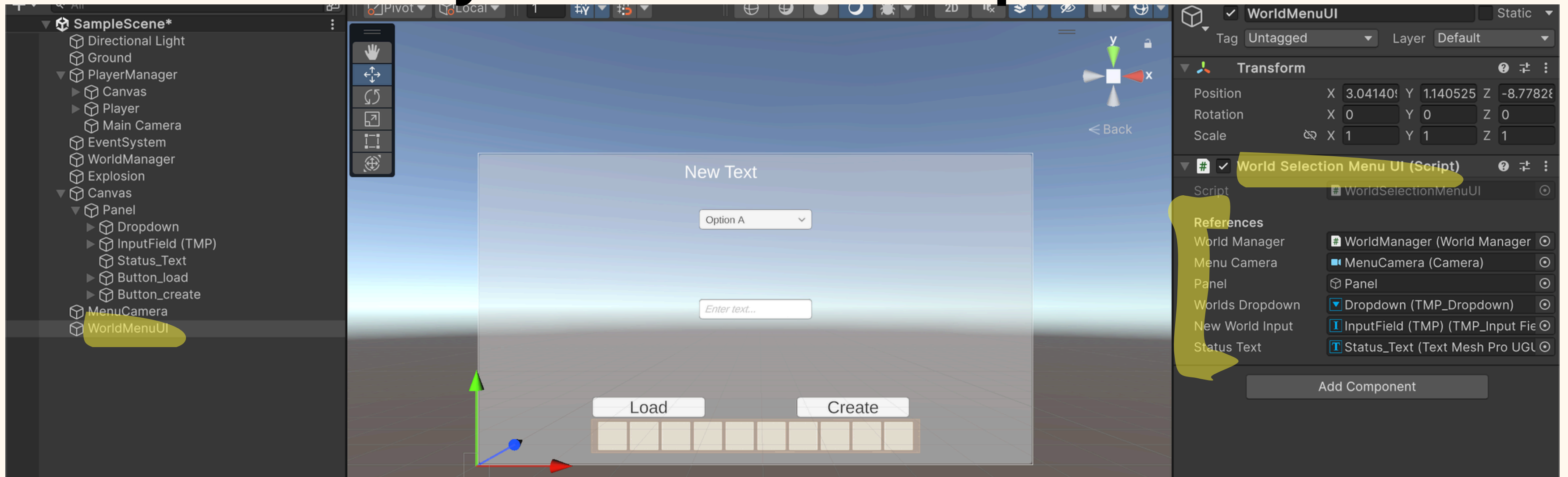
```
public void SaveWorld()
{
    if (!worldInitialized)
        return;

    chunkSpawner.SaveWorld(SavePath, player);
}

void OnApplicationQuit()
{
    if (worldInitialized)
        SaveWorld();
}

public bool IsWorldInitialized => worldInitialized;
```

Unity: Brancher Le Script Menu



Dans Unity, crée l'objet vide WorldMenuUI et ajoute le script que tu vient de coder

- **Assigne les champs:**
- **World Manager -> WorldManager**
- **Menu Camera -> MenuCamera**
- **Panel -> Panel**
- **Worlds Dropdown -> Dropdown (TMP)**
- **New World Input -> InputField (TMP)**
- **Status Text -> Status_Text (TMP)**

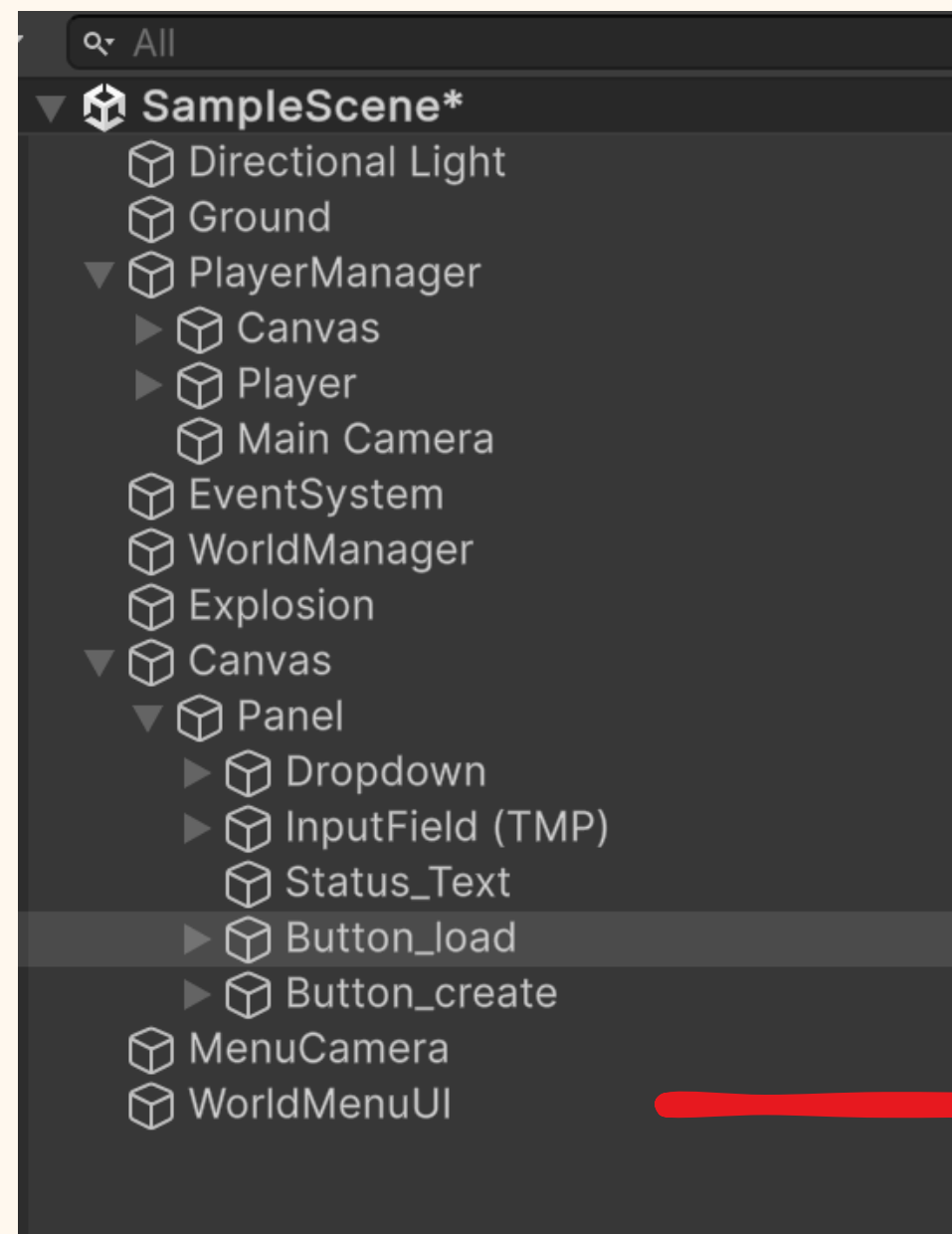
Brancher Les 2 Boutons

Button_load -> On Click -> objet WorldMenuUI ->

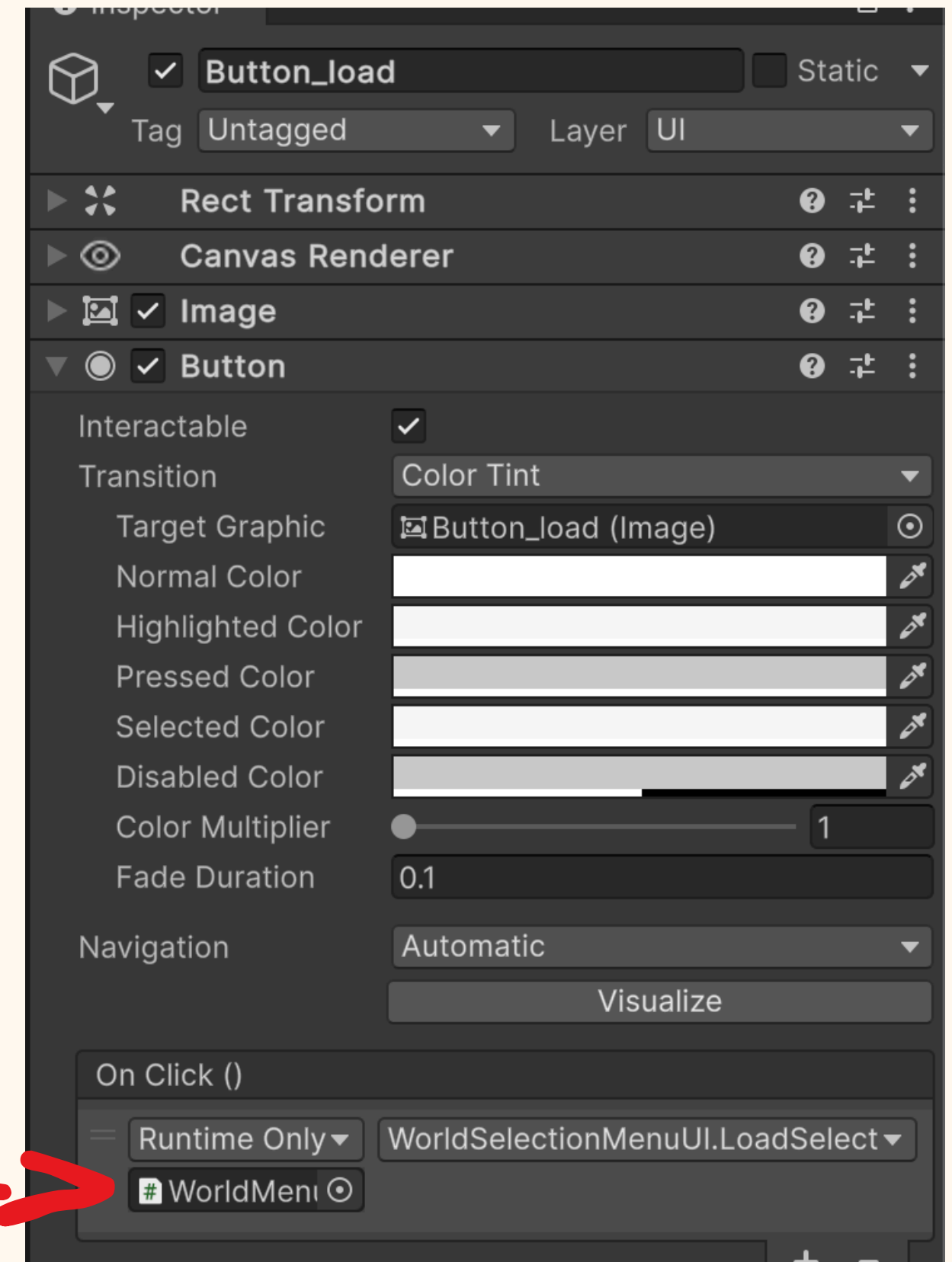
WorldSelectionMenuUI.LoadSelectedWorld()

Button_create -> On Click -> objet WorldMenuUI ->

WorldSelectionMenuUI.CreateAndStartWorld()



glisser



Dernier point

Ajoute ces lignes dans le chunk spawner

```
public void SaveWorld(string savePath, GameObject player)
{
    var save = CreateSaveData();

    if (player != null)
    {
        save.player = new PlayerData
        {
            position = player.transform.position,
            rotation = player.transform.rotation
        };
    }

    string directory = Path.GetDirectoryName(savePath);
    if (!string.IsNullOrEmpty(directory))
        Directory.CreateDirectory(directory);

    File.WriteAllText(savePath, JsonUtility.ToJson(save, true));
}

1 reference
private WorldSaveData LoadOrCreate(string savePath)
{
    WorldSaveData save;

    string directory = Path.GetDirectoryName(savePath);
    if (!string.IsNullOrEmpty(directory))
        Directory.CreateDirectory(directory);

    if (!File.Exists(savePath))
        save = new WorldSaveData { seed = Random.Range(0, 999999) };
    else
```


Maintenant des arbres !

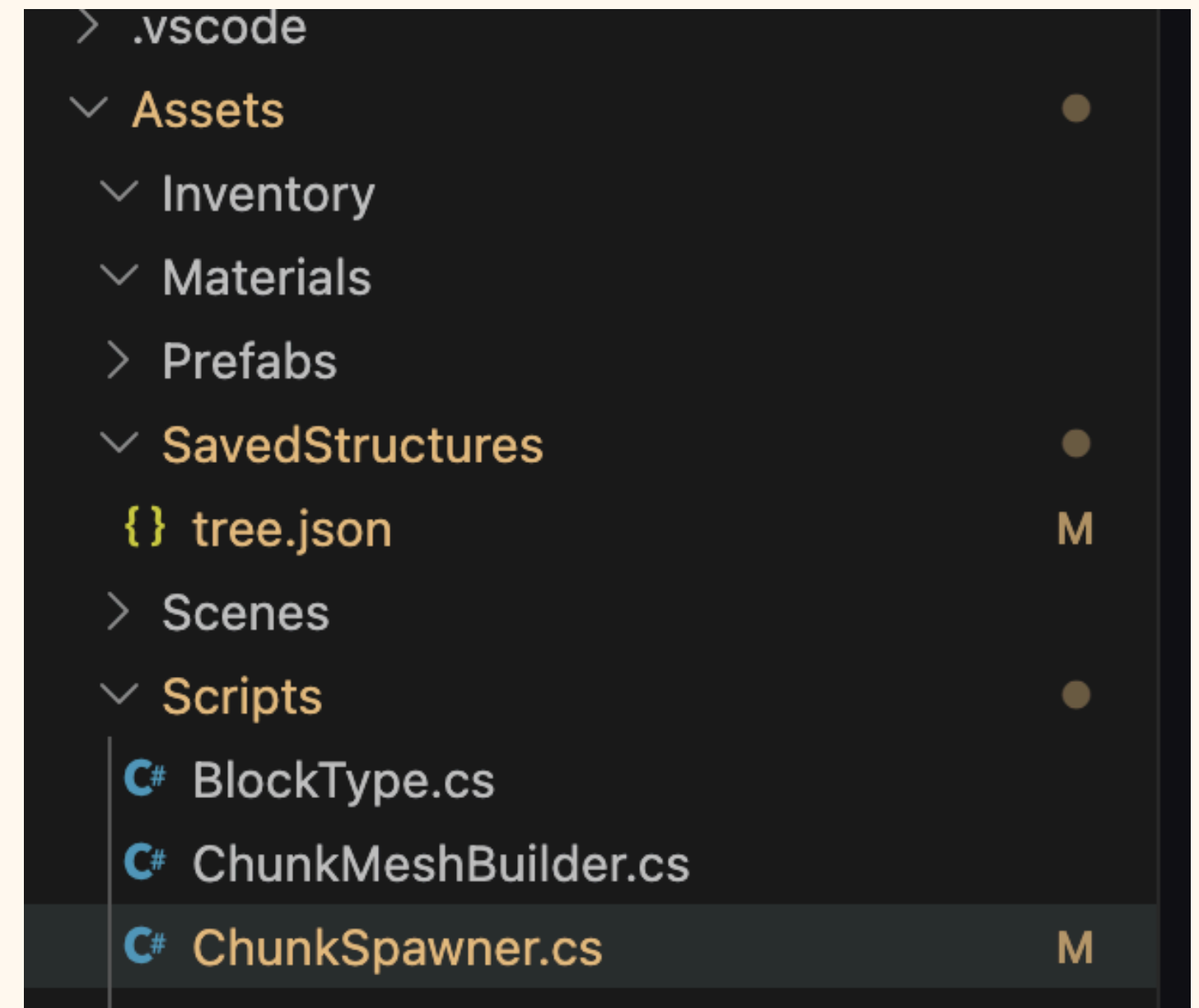


Objectif

- On veut générer des arbres automatiquement, de façon naturel et contrôlable.
- On utilise un modèle d'arbre dans tree.json, puis on le "copie-colle" à certains endroits du terrain.
- On veut pouvoir régler la quantité d'arbres sans modifier le code.

- Dans un premier temps, crée un nouveau dossier SavedStructures dans VS Code, puis crée un fichier tree.json.
- Copie-colle ensuite le contenu du fichier depuis ce lien :

<https://github.com/Ayly-EXE/LogiCube/blob/Cours7/Menu/Assets/SavedStructures/tree.json>



Ce qu'on ajoute dans ChunkSpawner

- On ajoute 4 idées:
 - Charger le modèle arbre depuis JSON.
 - Trouver des positions candidates dans chaque chunk.
 - Décider si un arbre peut apparaître.
 - Poser les blocs de l'arbre.

Ce qu'on va ajouter dans ChunkSpawner

- On ajoute 4 idées:
 - Charger le modèle arbre depuis JSON.
 - Trouver des positions candidates dans chaque chunk.
 - Décider si un arbre peut apparaître.
 - Poser les blocs de l'arbre.

Lire le JSON: classes de données

- Pour lire tree.json, Unity a besoin de classes "sérialisables".
- ajouter ceci au ChunkSpawner
- [System.Serializable] permet à JsonUtility de transformer le JSON en objets C#.
- type est l'ID du bloc (BlockType).
- x y z sont des offsets (positions relatives au centre de l'arbre).

```
[System.Serializable]
public class StructureBlockFileData
{
    public int type;
    public int x;
    public int y;
    public int z;
}

[System.Serializable]
public class StructureFileData
{
    public List<StructureBlockFileData> blocks = new();
}
```

Variable d'instances

On ajoute 2 réglages visibles dans Unity.

- treeFrequencyPercent = chance de spawn parmi les points testés.
- treeSpacing = distance entre points testés.
- Plus treeSpacing est grand, moins il y a d'arbres.

```
public class ChunkSpawner : MonoBehaviour
{
    25 references | ☑ Unity Serialized Field
    public WorldManagerScript worldManager;
    4 references | ☑ Unity Serialized Field
    public int viewDistanceInChunks = 1;
    [Header("Trees")]
    1 reference | ☑ Unity Serialized Field
    [Range(0, 100)] public int treeFrequencyPercent = 50;
    2 references | ☑ Unity Serialized Field
    [Min(1)] public int treeSpacing = 8;
```

```
2 references
private bool treeTemplateLoaded;
5 references
private readonly List<TreeBlockData> treeBlocks = new();
4 references
private int treeMinX;
4 references
private int treeMaxX;
4 references
private int treeMinZ;
4 references
private int treeMaxZ;

2 references
private struct TreeBlockData
{
    4 references | ☑ Unity Serialized Field
    public Vector3Int offset;
    2 references | ☑ Unity Serialized Field
    public BlockType type;
}
```


Chargement du template

On charge tree.json une seule fois au démarrage du monde.

ajoute LoadTreeTemplateIfNeeded() dans initialize()

```
public WorldSaveData Initialize(WorldManagerScript manager, string savePath)
{
    worldManager = manager;
    LoadTreeTemplateIfNeeded();
    WorldSaveData save = LoadOrCreate(savePath);
    RefreshVisibleChunks(true);
    return save;
}
```


LoadTreeTemplateIfNeeded() 1

Cette fonction ouvre tree.json, vérifie qu'il existe, puis lit les blocs.

```
1 reference
private void LoadTreeTemplateIfNeeded()
{
    if (treeTemplateLoaded)
        return;

    treeTemplateLoaded = true;
    treeBlocks.Clear();

    string path = Path.Combine(Application.dataPath, "SavedStructures", "tree.json");
    if (!File.Exists(path))
    {
        Debug.LogWarning("ChunkSpawner: tree.json not found at " + path);
        return;
    }

    StructureFileData file = JsonUtility.FromJson<StructureFileData>(File.ReadAllText(path));
    if (file == null || file.blocks == null || file.blocks.Count == 0)
    {
        Debug.LogWarning("ChunkSpawner: tree.json is empty.");
        return;
    }
}
```

LoadTreeTemplateIfNeeded() 2

On convertit chaque bloc JSON en bloc interne du jeu.

On calcule aussi la taille de l'arbre (min/max X/Z) pour ne pas le couper aux bords du chunk.

N'oublie pas que tu peux toujours copier-coller la fonction depuis GitHub :)

```
bool hasAny = false;

foreach (var b in file.blocks)
{
    if (!System.Enum.IsDefined(typeof(BlockType), b.type))
        continue;

    BlockType type = (BlockType)b.type;
    if (type == BlockType.Air)
        continue;

    Vector3Int offset = new Vector3Int(b.x, b.y, b.z);
    treeBlocks.Add(new TreeBlockData { offset = offset, type = type });

    if (!hasAny)
    {
        treeMinX = treeMaxX = offset.x;
        treeMinZ = treeMaxZ = offset.z;
        hasAny = true;
    }
    else
    {
        treeMinX = Mathf.Min(treeMinX, offset.x);
        treeMaxX = Mathf.Max(treeMaxX, offset.x);
        treeMinZ = Mathf.Min(treeMinZ, offset.z);
        treeMaxZ = Mathf.Max(treeMaxZ, offset.z);
    }
}

if (!hasAny)
{
    treeBlocks.Clear();
    Debug.LogWarning("ChunkSpawner: tree.json has no usable blocks.");
}
```

GenerateChunkBlocks()

Ajoute cette ligne dans la fonction :
on génère le terrain, puis on place un arbre.

```
1 reference
private Dictionary<Vector3Int, BlockType> GenerateChunkBlocks(Vector2Int chunkCoord)
{
    var chunkBlocks = new Dictionary<Vector3Int, BlockType>();
    Vector2 offset = GetSeedOffset();
    int half = worldManager.generator.chunkSize / 2;
    int startX = chunkCoord.x * worldManager.generator.chunkSize - half;
    int startZ = chunkCoord.y * worldManager.generator.chunkSize - half;

    for (int x = startX; x < startX + worldManager.generator.chunkSize; x++)
        for (int z = startZ; z < startZ + worldManager.generator.chunkSize; z++)
        {
            int height = GetTerrainHeight(x, z, offset);

            for (int y = 0; y <= height; y++)
                chunkBlocks[new Vector3Int(x, y, z)] = GetBlockTypeForHeight(y, height);
        }

    AddTreesToChunk(chunkBlocks, startX, startZ, offset);

    return chunkBlocks;
}
```

AddTreesToChunk()

Cette fonction parcourt des centres possibles d'arbres dans le chunk.

Elle vérifie chaque centre avec `ShouldPlaceTreeAt`, puis pose l'arbre avec `PlaceTree`.

La partie `centerMin/Max` empêche les arbres coupés.

```
private void AddTreesToChunk(Dictionary<Vector3Int, BlockType> chunkBlocks, int startX, int startZ, Vector2 offset)
{
    if (treeBlocks.Count == 0)
        return;

    int size = worldManager.generator.chunkSize;
    int endX = startX + size - 1;
    int endZ = startZ + size - 1;

    int centerMinX = startX - treeMinX;
    int centerMaxX = endX - treeMaxX;
    int centerMinZ = startZ - treeMinZ;
    int centerMaxZ = endZ - treeMaxZ;

    if (centerMinX > centerMaxX || centerMinZ > centerMaxZ)
        return;

    for (int x = centerMinX; x <= centerMaxX; x++)
        for (int z = centerMinZ; z <= centerMaxZ; z++)
        {
            int groundHeight = GetTerrainHeight(x, z, offset);
            if (!ShouldPlaceTreeAt(x, z, groundHeight))
                continue;

            PlaceTree(chunkBlocks, x, groundHeight, z);
        }
}
```

ShouldPlaceTreeAt()

Cette fonction répond à une seule question :
“Est-ce qu’on met un arbre à cette position ?”

Elle vérifie 3 choses :

1. Le sol ne doit pas être trop bas.
2. On ne teste pas toutes les cases, seulement des cases espacées.
3. On garde seulement une petite partie des cases testées.

on fabrique ensuite un hash sur base des coordonnées

```
private bool ShouldPlaceTreeAt(int x, int z, int groundHeight)
{
    if (groundHeight <= 3) return false;
    if (Mathf.Abs(x) % treeSpacing != 0 || Mathf.Abs(z) % treeSpacing != 0) return false;

    int hash = Mathf.Abs(x * 17 + z * 31 + seed);
    return (hash % 100) < treeFrequencyPercent;
}
```


ShouldPlaceTreeAt()

C'est quoi un hash, en général ?

=> une façon de transformer une information en un nombre ou une petite valeur.

À quoi ça sert ?

Un hash sert souvent à :

- reconnaître une donnée
- comparer rapidement des données
- ranger ou retrouver des informations
- fabriquer un résultat qui dépend toujours de la même entrée

Dans notre cas

On ne s'en sert pas pour sécuriser quelque chose.

On s'en sert pour transformer :

- la position x
- la position z
- le seed du monde

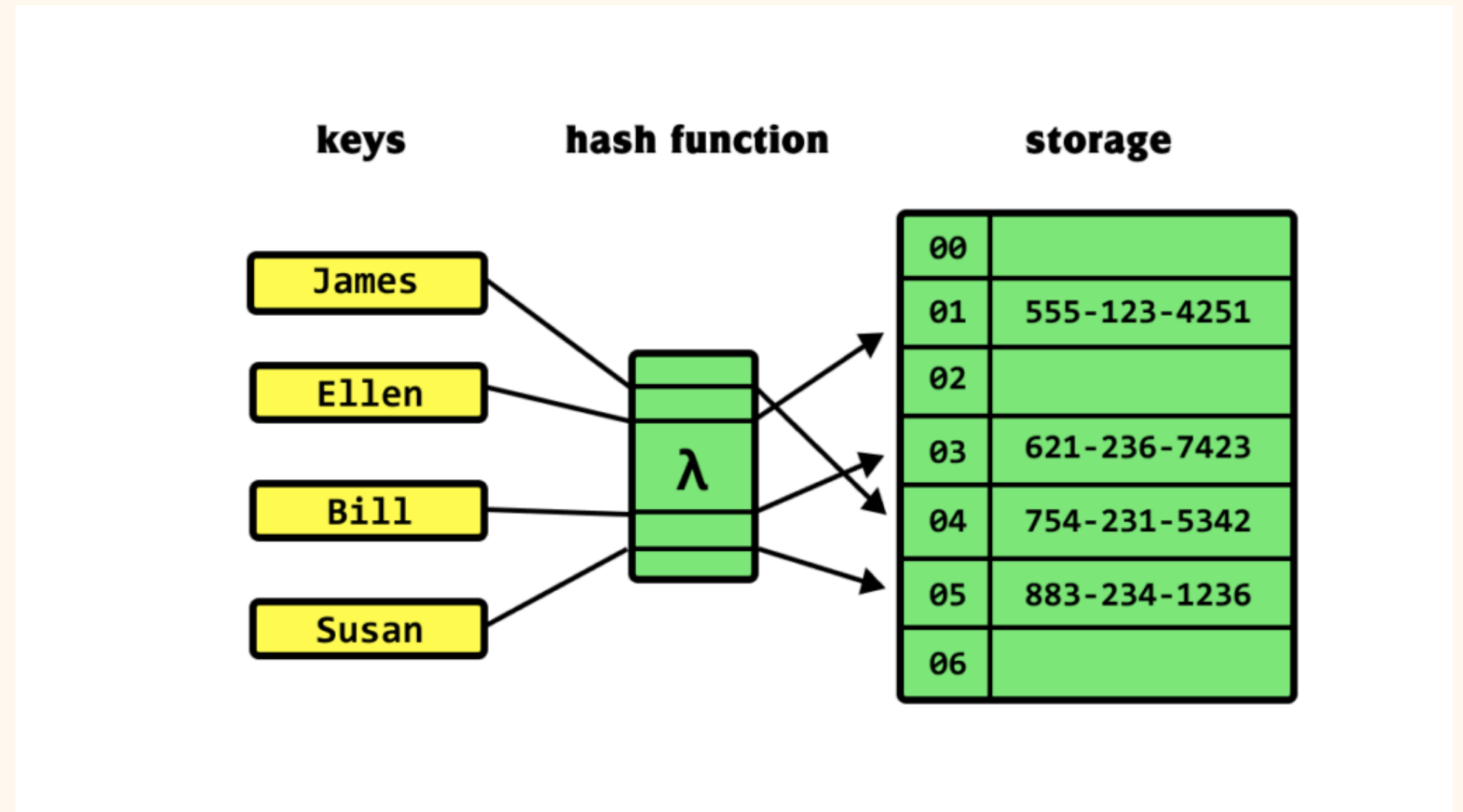
en un nombre qui semble aléatoire

Pourquoi c'est utile ici ?

Parce qu'on veut :

- un résultat différent selon l'endroit
- un résultat différent selon le monde
- mais un résultat qui reste le même quand on recharge

Un hash, c'est une "machine" qui prend une entrée et fabrique toujours le meme output



PlaceTree

On prend chaque bloc du modèle tree.json, on ajoute le décalage du centre choisi, et on écrit le bloc dans le chunk.

```
private void PlaceTree(Dictionary<Vector3Int, BlockType> chunkBlocks, int centerX, int groundHeight, int centerZ)
{
    foreach (var treeBlock in treeBlocks)
    {
        Vector3Int pos = new Vector3Int(
            centerX + treeBlock.offset.x,
            groundHeight + treeBlock.offset.y,
            centerZ + treeBlock.offset.z
        );

        chunkBlocks[pos] = treeBlock.type;
    }
}
```